

Git: configuração e fluxo de trabalho local

Introdução

Configuração do repositório, revisão de alterações e organização do histórico com boas práticas de commits.

INTEGRANTES

Arthur Entsev • Eduardo Kioshi
Gustavo Henrique • Thiago Pereira

git init

O QUE É

Comando que inicia um repositório Git na pasta atual.

O QUE FAZ

Cria a pasta .git, onde o Git mantém o histórico e os metadados do repositório.

```
git init
```

Executar dentro da pasta do projeto. Os arquivos ainda não viram commits.

git config: nome e e-mail

O QUE É

Comando que consulta ou altera as configurações do Git.

NOME E E-MAIL DO USUÁRIO

user.name define o nome do autor; user.email define seu e-mail. Esses dados identificam a autoria dos commits.

```
git config --local user.name "Nome do Grupo"  
git config --local user.email "grupo@example.com"
```

git config: local × global

GLOBAL

Padrão para os repositórios do usuário nesta máquina.

LOCAL

Vale só no repositório atual e prevalece sobre o global.

```
git config --global user.name "Seu Nome"
git config --global user.email "voce@example.com"
git config --local user.name "Nome do Grupo"
git config --local user.email "grupo@example.com"
```

.gitignore: o que é e por que usar

O QUE É

Arquivo de texto com padrões que indicam quais arquivos não rastreados devem ser ignorados pelo Git.

POR QUE USAR

Evita incluir dependências locais, configurações privadas e logs por acidente. Mantém o histórico focado no código necessário ao projeto. O próprio .gitignore é versionado com o grupo.

Não deixa de rastrear arquivos que já foram commitados.

.gitignore: exemplos de padrões

```
node_modules/  
.env  
*.log
```

node_modules/

Ignora a pasta de dependências.

.env

Ignora o arquivo de configuração privada.

*.log

Ignora arquivos como app.log e debug.log.

Barra final indica diretório. O asterisco * funciona como curinga.

git diff

VER ALTERAÇÕES ANTES DE COMMITAR

Mostra as mudanças nos arquivos rastreados ainda não preparados, permitindo revisar o conteúdo antes do commit.

```
git diff -- assets/style.css
```

```
- background: #ffcc45;  
+ background: #48e5c2;
```

O sinal - mostra a linha anterior; o sinal + mostra a nova.

Depois de git add, usar git diff --cached para revisar o que foi preparado.

Boas práticas de mensagens de commit

MÁ MENSAGEM

salvando tudo

É vaga: não informa qual mudança foi feita
nem ajuda a localizá-la no histórico.

BOA MENSAGEM

feat(ui): altera cor do botao principal

É curta, específica e descreve a mudança registrada.

Mensagens: Conventional Commits

Convenção de escrita que dá uma estrutura às mensagens.
O escopo é opcional; no exemplo, ui indica a interface.

```
tipo(escopo opcional): descricao
```

feat: funcionalidade

fix: correção

docs: documentação, se adotado pela equipe

O grupo combina o padrão e o aplica de forma consistente.

git commit --amend

CORRIGIR O ÚLTIMO COMMIT

Recria o último commit. Pode corrigir sua mensagem e também incluir alterações que já estejam preparadas.

```
git commit -m "salvando tudo"  
git commit --amend -m "feat: esclarece texto do botao"
```

A mensagem vaga é substituída pela descrição da alteração.
O identificador muda porque o Git cria uma nova versão do commit.

Se o commit já foi compartilhado, combine a reescrita com a equipe.

Organização de commits

COMMITTS PEQUENOS E ATÔMICOS

Cada commit reúne mudanças relacionadas a um único objetivo. Um objetivo pode envolver vários arquivos.

1 Criar a página inicial

2 Alterar a cor do botão

3 Esclarecer o texto do botão

COMMITTS GENÉRICOS: "SALVANDO TUDO"

Misturar mudanças sem relação e usar uma mensagem vaga dificulta a revisão. Separar por objetivo facilita entender e localizar alterações.